

End of Semester Report - Intrusion Detection Systems

Daniel Broyd

July 13, 2025

1 Features Used in the ML Technique

The implementation uses a combination of categorical and numerical features extracted from network traffic logs for intrusion detection:

Selected Features:

Table 1: Feature Descriptions

Feature	Explanation	Relevance	Extraction
Computer	Identifies the specific machine in the network.	Targets may focus on hosts or show multi-host patterns.	Raw log $\rightarrow$ one-hot encode.
DestinationPortName	Indicates the service being accessed.	Attacks target services (e.g., HTTP on 80).	Raw log $\rightarrow$ one-hot encode.
EventID	Windows logs categorize events by ID.	Attack patterns map to specific event types.	Raw log $\rightarrow$ one-hot encode.
EventRecordID	Numeric identifier for individual records.	Tracks sequence and timing patterns.	Raw log $\rightarrow$ MinMax scaling.
Execution_ProcessID	ID of process initiated during the event.	Anomalous behavior can include running specific processes.	Raw log $\rightarrow$ MinMax scaling.
Initiated	Boolean for connection initiation direction.	Distinguishes inbound vs. outbound.	Raw log $\rightarrow$ one-hot encode.
ProcessId	Process ID related to the event.	Correlates events to processes.	Raw log $\rightarrow$ MinMax scaling.
SourceIsIpv6	Boolean indicating IP version.	Some attacks target specific protocols.	Raw log $\rightarrow$ one-hot encode.
Temporal	<i>Y/M/W/Dow</i> one-hot; <i>D/H/M</i> MinMax.	Captures timing patterns of attacks.	Extracted from SystemTime.

The features were defined in the reference paper in Table 2. The paper provided a general description of the feature categories but did not provide detailed justification for each individual feature.

2 ML Technique Parameters

The implemented model is a Stacked Autoencoder with a symmetric hourglass architecture: the encoder compresses input $\rightarrow 50 \rightarrow 30 \rightarrow 20 \rightarrow 10 \rightarrow 5$ (latent), and the decoder

mirrors this as $5 \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow 50 \rightarrow \text{output}$. All hidden layers employ ReLU activations, the output layer uses Sigmoid, batch normalization is applied after each dense layer, and Binary Cross-Entropy loss is minimized.

Training is performed with batch size 50 over 20 epochs, using the Adam optimizer with learning rate $1\text{e-}3$ (vs. the paper’s $1\text{e-}2$) and weight decay $1\text{e-}5$.

Compared to the original parameters (table 6 in the paper), I lowered the learning rate for stability (no convergence otherwise) and removed the paper’s 0.15 dropout. While dropout normally improves generalization towards unseen data, I intentionally omitted it so that the autoencoder maintains higher reconstruction error on unseen, attack-like samples, enhancing anomaly detection sensitivity.

3 ML Technique Implementation Details

The Stacked Autoencoder was implemented using PyTorch, with a symmetric encoder-decoder architecture, can utilize a GPU if available.

Classification Methodology

The classification methodology begins with a training phase in which the autoencoder is trained exclusively on normal traffic data to learn the underlying patterns of “normal” behavior and achieve low reconstruction error on such samples. Once trained, for any input sample $\mathbf{x}$ the reconstruction error is calculated as

$$\text{Error}(\mathbf{x}) = \text{MSE}(\mathbf{x}, \hat{\mathbf{x}}) = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2,$$

where $\hat{\mathbf{x}}$ is the output reconstructed by the autoencoder. A threshold τ is then determined—typically set at the 99th percentile of reconstruction errors on validation (normal) data—to separate normal from anomalous behavior. Finally, a sample is classified as an attack if its reconstruction error exceeds this threshold, otherwise it is deemed normal:

$$\text{Classification}(\mathbf{x}) = \begin{cases} \text{Attack}, & \text{Error}(\mathbf{x}) > \tau, \\ \text{Normal}, & \text{Error}(\mathbf{x}) \leq \tau. \end{cases}$$

4 ML Technique Training and Testing Details

Training Process: I split the normal dataset 80%/20% for training/testing, train on the 80% (only unlabeled, normal data), and use batches of 50 (shuffled) samples per iteration. After each epoch the model is checkpointed and the average loss is recorded.

Testing Process: I evaluate on the held-out normal set and attack data (EORS and EOHT), computing reconstruction errors via MSE. A threshold is then set at the 99th percentile of normal-data errors, and detection accuracy and false positive rates are calculated afterwards.

5 Results Comparison

Paper vs. My Implementation

The original paper reported $\approx 95\%$ detection accuracy with $< 1\%$ false positive rate. My implementation achieved an average reconstruction error of 0.0006 on normal data versus 0.0686 on attack data, with 97.37% detection accuracy (497,645/511,105), a 1.00% false positive rate (3,266/326,581), and a 99th-percentile threshold of 0.0040 - an improvement of the paper results.

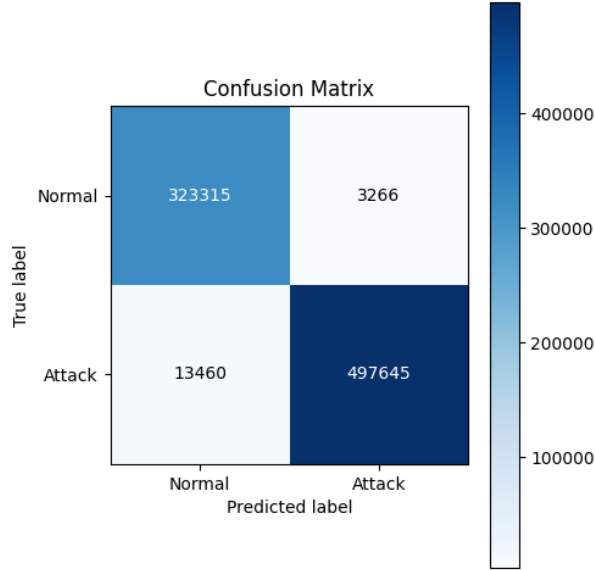


Figure 1: Confusion Matrix of Detection Results on Unseen Data

This improvement could probably be explained by the removal of dropout to ensure higher reconstruction error on unseen, attack-like samples, thereby enhancing anomaly detection sensitivity (as mentioned in section 2).

6 Analysis of Detection Mistakes

Detection errors fall into two main categories:

1. False Negatives (Missed Attacks)

- **Low-intensity Attacks:** subtle anomalies whose reconstruction error stays below the detection threshold.
- **Edge Cases:** events at the boundary of the normal-behavior distribution that the model misclassifies.

2. False Positives (Normal Traffic Flagged as Attacks)

- **Rare Normal Patterns:** legitimate traffic shapes that were underrepresented during training.
- **Bursty Traffic:** sudden but non-malicious spikes in network usage that trigger alarms.

The mistakes primarily stem from the inherent challenge of defining a clear boundary between normal and anomalous behavior. The reconstruction error threshold represents a trade-off between detection sensitivity and false alarm rate.

7 Potential Improvements

The primary inherent limitation of the autoencoder approach is its unsupervised nature - it can only learn what "normal" data patterns look like and flags deviations from these patterns. Several approaches could improve detection accuracy (while staying on the autoencoder path):

1. **Dynamic Thresholding:** Implement adaptive thresholds that adjust based on network conditions and time of day rather than using a fixed threshold
2. **Feature Engineering:**
 - Implement sequence-based features to capture temporal patterns better
 - Apply more sophisticated feature selection methods
3. **Ensemble Approach:** Combine the autoencoder with other detection methods:
 - Use supervised learning for known attack patterns
 - Complement with rule-based detection for well-defined attack signatures
 - Implement multiple autoencoders specialized for different traffic types

Report on Variational Autoencoder (VAE) for Intrusion Detection

ML Technique Description

A Variational Autoencoder (VAE) is an unsupervised, generative model that improves upon the standard autoencoder. Its encoder learns a probabilistic distribution (e.g., a Gaussian with mean and variance) in the latent space, rather than a single point. The decoder then samples from this distribution to reconstruct the input, enabling the model to also generate new, similar data.

Rationale

For intrusion detection, a VAE is effective because it models the probability distribution of "normal" traffic, creating a robust representation that is less sensitive to minor variations. Attacks, being outliers to this distribution, will generate high reconstruction errors. The VAE's use of KL divergence as a regularizer also organizes the latent space, which improves the separation between normal and anomalous data and enhances the detection of novel threats.

Features Used in the ML Technique

For the VAE model, I will utilize the exact same set of features as those used for the Stacked Autoencoder in the first part of this project. This ensures consistency and allows for a direct and fair comparison of the models' performance. The selected features, a mix of categorical and numerical data from network traffic logs, provide a comprehensive view of network activity, capturing information about the event's origin (**Computer**), target (**Destination PortName**), nature (**EventID**), and timing (**Temporal Features**).

The feature extraction and preprocessing will mirror the methodology from Part 1. **Categorical Features** (**Computer**, **Destination PortName**, **EventID**, **Initiated**, **SourceIsIpv6**, and parts of **Temporal Features**) will be one-hot encoded. **Numerical Features** (**EventRecordID**, **Execution ProcessID**, **ProcessId**, and parts of **Temporal Features**) will be scaled to a range between 0 and 1 using a **MinMaxScaler**. This normalization is critical for neural network models.

ML Technique Implementation Details

The VAE will be implemented using PyTorch. The core of the implementation involves three main parts: an encoder, a decoder, and a reparameterization trick to allow for backpropagation through the sampling process.

- **Encoder:** It will take the preprocessed input vector and pass it through several dense layers to output two vectors: a mean vector (μ) and a log-variance vector ($\log(\sigma^2)$).
- **Reparameterization Trick:** To sample from the learned distribution $N(\mu, \sigma^2)$ in a way that allows gradient flow, a latent vector z is created as $z = \mu + \sigma \odot \epsilon$, where ϵ is sampled from a standard normal distribution $N(0, I)$.

- **Decoder:** This network will take the sampled latent vector z and reconstruct the original input data through a series of dense layers, symmetric to the encoder.
- **Loss Function:** The VAE’s loss function is the sum of the reconstruction loss (MSE) and the KL divergence, which regularizes the latent space.

$$\mathcal{L}(x, \hat{x}) = \text{MSE}(x, \hat{x}) - \frac{1}{2} \sum (1 + \log(\sigma^2) - \mu^2 - \sigma^2) \quad (1)$$

Classification will be performed similarly to the standard autoencoder: the model is trained only on normal data, and a threshold is set on the reconstruction error to distinguish between normal and attack traffic.

ML Technique Parameters

The parameters for the VAE will be chosen to be comparable to the Stacked Autoencoder to facilitate a meaningful comparison, with adjustments for the VAE’s specific architecture.

- **Architecture:** The encoder and decoder will have a symmetric structure: Input $\rightarrow 50 \rightarrow 30 \rightarrow$ (Latent Space: 10-dim μ , 10-dim $\log \sigma^2$). The decoder will mirror this: Latent (10-dim) $\rightarrow 30 \rightarrow 50 \rightarrow$ Output. A 10-dimensional latent space is chosen as a starting point to balance information compression and representational capacity.
- **Activation Functions:** ReLU will be used for all hidden layers, with a Sigmoid function for the final output layer, as the input data is scaled between 0 and 1.
- **Optimizer:** The Adam optimizer will be used with a **learning rate** of 1e-3.
- **Batch Size:** A batch size of 50 will be used, consistent with the previous experiment.
- **Epochs:** The model will be trained for 20 epochs.

Report on Random Forest for Intrusion Detection

ML Technique Description

Random Forest is an ensemble machine learning technique that constructs a multitude of decision trees during training and outputs the class (for classification) of the individual trees, thereby improving predictive accuracy and controlling overfitting. By using bootstrap aggregation (bagging) to train each tree on a random subset of the data and by selecting a random subset of features at each split, Random Forest introduces diversity among trees, enhances robustness to noise and outliers, and provides estimates of feature importance for interpretability.

Rationale

Random Forest is highly effective for intrusion detection as it is fundamentally a classification problem. Its effectiveness stems from its high accuracy on complex datasets, robustness to overfitting, and its ability to handle a large number of features. A key advantage is its capability to provide a reliable measure of feature importance, which can help security analysts understand which network attributes are most indicative of an attack. As a supervised method, it can explicitly learn the patterns that differentiate known attacks from normal traffic, making it very powerful for detecting threats that have been seen before.

Features Used in the ML Technique

I will use the same set of features as in the previous models to maintain consistency. For a supervised model like Random Forest, this rich feature set allows the algorithm to identify the rules that distinguish malicious activity from normal traffic. The key difference is that the dataset will be labeled with 'normal' and 'attack' classes for supervised training (rather than training on only normal data).

The feature extraction and pre-processing will be identical to the methods used for the autoencoder models. Categorical features will be one-hot encoded, and numerical features will be scaled using `MinMaxScaler`. While tree-based models are less sensitive to feature scaling, maintaining a consistent pipeline is best practice for a fair comparison.

ML Technique Implementation Details

The Random Forest model will be implemented using the `scikit-learn` library in Python.

- **Data Splitting:** The full labeled dataset will be split into a training set (80%) and a testing set (20%).
- **Training:** The `RandomForestClassifier` will be trained on the training set, building a specified number of decision trees on different random samples of the data and features.
- **Prediction:** Once trained, the model will predict the class of each sample in the testing set by taking a majority vote from all decision trees in the forest.
- **Evaluation:** Performance will be evaluated using standard classification metrics such as accuracy, precision, recall, F1-score, and a confusion matrix.

ML Technique Parameters

The parameters for the Random Forest model will be chosen based on common practices that balance performance and computational cost.

- **n_estimators:** 100. This is the number of trees in the forest, a value that generally provides a good balance between performance and training time.
- **max_depth:** None. By default, trees are grown until all leaves are pure. This allows them to capture complex patterns in the data.
- **min_samples_split:** 2. The minimum number of samples required to split an internal node.
- **min_samples_leaf:** 1. The minimum number of samples required to be at a leaf node.
- **criterion:** 'gini'. The Gini impurity is a standard and computationally efficient metric for measuring split quality.